

Induction Proof Backend: Simple Guide

This document explains, in plain language, how the backend now supports Induction proofs in a way similar to the existing Equational Reasoning (Racket) mode. You do not need prior experience with React, HTML, or Django to follow this.

What This Adds

- New endpoints that the frontend can call to build, edit, and validate induction proofs.
- A server-side "Induction Proof" object that keeps track of two proof phases:
 - Base Case (sometimes called "anchor" case)
 - Leap Step (sometimes called "inductive step")
- Each phase is a two-sided proof with a Left-Hand Side (LHS) and Right-Hand Side (RHS), just like Racket mode.
- Simple ways to set the starting statements (goals/premises), apply rules, do substitutions, and delete the last step.

Key Ideas (No Jargon)

- **Case:** Which part of the induction you are working on.
 - **base** (a.k.a. "anchor")
 - **leap** (a.k.a. "inductive step")
- **Side:** Which half of the proof you are editing.
 - **LHS** (Left-Hand Side)
 - **RHS** (Right-Hand Side)
- **Goal/Premise:** The starting expression for LHS or RHS.
- **Rule Application:** A backend operation that transforms the previous expression into a new one (following proof rules).
- **Substitution:** Like a rule, but you supply an extra expression to plug in.
- **IH (Induction Hypothesis):** The equality you assume holds at step **k** and use during the leap step from **k** to **k+1**.

Where The Code Lives

- Endpoints and induction wiring are implemented in [django_server/induction_api/views.py](#).
- Routes are configured in [django_server/induction_api/urls.py](#).
- The induction engine (data structures and rule logic) already existed in [django_server/expression_tree/IndProofs.py](#) and related files.

New Endpoints

These endpoints are designed to be called by the frontend. You can also call them with tools like Postman or **curl** for testing.

1. Initialize an induction proof

- Method: **POST**

- Path: `/api/v1/induction/set-current-proof`
- Purpose: Builds the induction proof in the server with your parameters and (optionally) function definitions.
- Request body (example):

```
{
  "struct": "int",
  "ivar": "n",          // induction variable
  "aval": "0",          // anchor value
  "lvar": "k",          // leap variable
  "lhsPremise": "(sum n)",
  "rhsPremise": "(* n (+ n 1) (/ 1 2))",
  "definitions": [
    { "label": "(f n)", "type": "int -> int", "expression": "..." }
  ]
}
```

- What it returns: JSON with the latest racket string and `jsonTree` for both base case and leap step, LHS and RHS.

2. Apply a rule to the current case/side

- Method: `POST`
- Path: `/api/v1/induction/apply-rule`
- Purpose: Applies a rule to the last expression for either base or leap, LHS or RHS.
- Request body (example):

```
{
  "case": "base",        // "base" or "leap"
  "side": "LHS",         // "LHS" or "RHS"
  "currentRacket": "(sum 0)",
  "rule": "rewrite math",
  "startPosition": 7
}
```

- Response includes: `isValid`, `racket`, `jsonTree`, and `lineNum`.

3. Delete the last line in the current case/side

- Method: `DELETE`
- Path: `/api/v1/induction/delete-line/{case}/{side}`
 - Example: `/api/v1/induction/delete-line/base/LHS`
- Purpose: Removes the most recent proof step on the selected case and side.

4. Set or reset a goal/premise

- Method: `POST`
- Path: `/api/v1/induction/check-goal`

- Purpose: Clears previous lines and sets a new starting expression for the selected case and side.
- Request body (example):

```
{
  "case": "base",
  "side": "RHS",
  "goal": "(* 0 (+ 0 1) (/ 1 2))"
}
```

5. Apply a substitution (like a rule, but with an extra expression)

- Method: **POST**
- Path: **/api/v1/induction/substitution**
- Purpose: Same as **apply-rule** but also includes a **substitution** string.
- Request body (example):

```
{
  "case": "leap",
  "side": "RHS",
  "currentRacket": "(* (+ k 1) (+ (+ k 1) 1) (/ 1 2))",
  "rule": "rewrite math",
  "startPosition": 5,
  "substitution": "(+ k 1)"
}
```

How This Mirrors Racket Mode

- Racket mode worked with a two-sided proof (**LHS/RHS**) and applied rules to transform the expression step-by-step.
- Induction mode uses the same idea, but adds a **case** selector (**base** vs **leap**).
- In the older text file tests, a line **-1** meant "switch" (e.g., stop reading more steps). In the app, you simply send **case** and **side** in the request instead of relying on a sentinel value.

Typical Usage Flow

1. Call **set-current-induction-proof** with your induction parameters. This seeds:
 - Base case LHS/RHS: replaces **ivar** with **aval**.
 - Leap step LHS/RHS: replaces **ivar** with **(+ lvar 1)** (for integer induction).
 - Induction Hypothesis **IH**: built from your premises with **ivar → lvar** and made available as a rule.
2. For each side (**LHS** or **RHS**) and case (**base** or **leap**), apply rules via **apply-rule** or **substitution**.
3. If you need to start over on a side, call **check-goal** with a new **goal**.
4. If you make a mistake on the last step, call **delete-line/{case}/{side}**.

Quick Test Examples (Curl)

Initialize (Base + Leap seeded, IH registered):

```
curl -X POST http://localhost:8000/api/v1/induction/set-current-proof \
-H "Content-Type: application/json" \
-d '{
  "struct": "int",
  "ivar": "n",
  "aval": "0",
  "lvar": "k",
  "lhsPremise": "(sum n)",
  "rhsPremise": "(* n (+ n 1) (/ 1 2))",
  "definitions": []
}'
```

Apply a rule on base/LHS:

```
curl -X POST http://localhost:8000/api/v1/induction/apply-rule \
-H "Content-Type: application/json" \
-d '{
  "case": "base",
  "side": "LHS",
  "currentRacket": "(sum 0)",
  "rule": "rewrite math",
  "startPosition": 7
}'
```

Set a new goal (base/RHS):

```
curl -X POST http://localhost:8000/api/v1/induction/check-goal \
-H "Content-Type: application/json" \
-d '{
  "case": "base",
  "side": "RHS",
  "goal": "(* 0 (+ 0 1) (/ 1 2))"
}'
```

Delete last step (leap/LHS):

```
curl -X DELETE http://localhost:8000/api/v1/induction/delete-line/leap/LHS
```

Apply substitution (leap/RHS):

```
curl -X POST http://localhost:8000/api/v1/induction/substitution \
-H "Content-Type: application/json" \
```

```
-d '{
  "case": "leap",
  "side": "RHS",
  "currentRacket": "(* (+ k 1) (+ (+ k 1) 1) (/ 1 2))",
  "rule": "rewrite math",
  "startPosition": 5,
  "substitution": "(+ k 1)"
}'
```

Server Setup Notes

- Ensure migrations are up to date, and that the cache table exists:

```
cd django_server
python manage.py makemigrations
python manage.py migrate
python manage.py createcachetable
```

- Run the server:

```
python manage.py runserver
```

Error Handling & Validation

- Responses include `isValid` and `errors` when something goes wrong (e.g., bad rule name or invalid expression).
- The backend tries to keep your proof data consistent and will report issues rather than crash.

About "IH" (Induction Hypothesis)

- When you initialize, the backend builds `IH` from your premises by replacing the induction variable (`ivar`) with the leap variable (`lvar`).
- `IH` is available as a rule you can apply in the leap step when the current expression matches either side of the hypothesis.

Differences for Lists (Future Work)

- The current initialization for leap uses `(+ lvar 1)` which is correct for integer induction.
- For list induction, the leap premise will use a list-specific successor (e.g., `cons`). This guide focuses on integer induction; list support can be added similarly.

In Short

- You choose the case (`base` or `leap`) and the side (`LHS` or `RHS`).
- You set a starting `goal` (or let initialization seed one for you).
- You apply rules or substitutions to build the proof line by line.

- The backend keeps track of everything and returns both a string and a structured `jsonTree` for the UI to display.

If you want help wiring a specific UI button click to the right endpoint call, just share the UI state you have and we'll map it to the correct `case`, `side`, and payload.